

这份讲义讲的是：不同类型的大语言模型，会用不同的“训练题目”来学习语言；而训练超大模型时，真正难的不只是定义目标函数，而是如何稳定、高效、省显存地把目标优化好。

通俗地说：

训练大模型就像训练学生。
先要设计“考试题目”：让它预测下一个词、补全被遮住的词、判断两句话关系、或者把输入文本转成输出文本。
然后要设计“学习方法”：怎么更新参数、怎么控制步子大小、怎么省内存、怎么避免训练崩掉。

1. 训练目标：模型到底在学什么？

讲义先区分了三类语言模型：

模型类型	代表模型	核心特点	通俗解释
Decoder-only	GPT-3	从左到右，一个 token 一个 token 地生成	像写作文，看到前文，预测下一个字
Encoder-only	BERT	双向理解整句话	像做阅读理解，看完整句子后理解每个词
Encoder-decoder	T5、BART	输入一段内容，输出另一段内容	像翻译、摘要、改写、问答

2. Decoder-only：GPT 类模型怎么训练？

Decoder-only 模型的训练目标是：

给定前面的 token，预测下一个 token。

例如：

the mouse ate the → cheese

模型要学会：

```
p(cheese | the mouse ate the)
```

也就是：在看到前文的情况下，下一个词最可能是什么。

讲义中把这个叫做 **autoregressive language modeling**，也就是自回归语言建模。

通俗解释：

GPT 的训练方式很简单粗暴：

把大量文本切成序列，然后不断问模型：“看到前面这些字，后面应该是什么？”

它答错了，就根据错误程度调整参数；答对得越多，语言能力越强。

它的目标函数是 **最大似然估计 Maximum Likelihood**，等价于最小化负对数似然：

让真实文本序列出现的概率尽可能高。

换句话说：

模型不是在“背知识点”，而是在学习：

什么样的上下文后面，什么样的词最合理。

3. Encoder-only: BERT 类模型怎么训练？

BERT 不负责直接生成文本，它更擅长做理解任务，比如分类、匹配、判断关系。

BERT 的训练目标主要有两个：

1. **Masked Language Modeling**，遮蔽语言建模
2. **Next Sentence Prediction**，下一句预测

3.1 Masked Language Modeling：把词遮住，让模型猜

例子：

```
[the, [MASK], ate, [MASK], cheese]
```

模型要预测原句是：

```
[the, mouse, ate, the, cheese]
```

通俗解释：

这就像语文考试里的完形填空。

给你一句话，把其中几个词挖掉，让你根据上下文猜出来。

BERT 不是只看左边，而是可以同时看左边和右边。所以它能学到更强的 **双向上下文表示**。

例如预测 [MASK] 时：

```
the [MASK] ate the cheese
```

它既能看见前面的 the ，也能看见后面的 ate the cheese 。

这和 GPT 不一样：

模型	能看到什么
GPT	只能看左边上下文
BERT	左右上下文都能看

所以 BERT 更适理解任务，GPT 更适合生成任务。

3.2 为什么 BERT 不是所有被选中的词都替换成 [MASK]?

讲义中特别提到，BERT 会随机选择 15% 的 token，其中：

操作	概率
替换成 [MASK]	80%
保持原词不变	10%
替换成随机词	10%

为什么这么麻烦？

因为如果训练时永远看到 [MASK] ，但真实使用时句子里没有 [MASK] ，就会出现 **训练和测试不一致**。

通俗解释：

如果学生平时练习题里永远有“_____”，但考试时没有空格，他可能反而不适应。所以训练时偶尔也让模型看到正常词，减少这种落差。

3.3 Next Sentence Prediction：判断两句话是否连续

BERT 还会做一个二分类任务：

句子 A 后面是否真的接着句子 B？

例如：

[CLS] the mouse ate the cheese [SEP] it was full → 1

表示第二句确实接在第一句后面。

[CLS] the mouse ate the cheese [SEP] hello world → 0

表示第二句是随机拼上去的。

其中 [CLS] token 的 embedding 会被用来做分类判断。

通俗解释：

这像是在训练模型理解“两句话之间有没有逻辑关系”。
不只是理解单个词，还要理解句子之间是否连贯。

不过讲义后面也提到，RoBERTa 后来去掉了 Next Sentence Prediction，因为实验发现它帮助不大。

4. BERT 的意义：NLP 从此进入“预训练 + 微调”时代

讲义强调，BERT 的重要性不只是模型本身，而是它改变了 NLP 的范式。

以前很多 NLP 任务要分别设计模型，比如情感分类、问答、文本匹配、自然语言推理等。

BERT 之后，大家开始采用：

先在大规模语料上预训练一个通用模型
再针对具体任务进行 **fine-tuning**

通俗解释：

先培养一个“语言基础很强的学生”，
然后让他再专门准备某一门考试，比如阅读理解、判断题、分类题。

这就是现在大模型训练的基本思想。

5. RoBERTa：对 BERT 的改进

RoBERTa 是对 BERT 的强化版，主要改动包括：

改动	含义
去掉 Next Sentence Prediction	发现这个目标帮助不大
使用更多数据	从 16GB 文本增加到 160GB 文本
训练更久	提升模型性能

通俗解释：

RoBERTa 的思路不是彻底换架构，而是说：
BERT 的方向是对的，但训练得还不够充分。
去掉没用的任务，多喂数据，多训练，效果就会显著提升。

6. Encoder-decoder：BART 和 T5 怎么训练？

Encoder-decoder 模型的核心是：

输入一段文本 → 输出另一段文本

这类模型既能理解输入，又能生成输出。

例如表格转文本：

[name: Clowns | eatType: coffee shop]
→
Clowns is a coffee shop.

通俗解释：

Encoder 负责“读懂输入”，Decoder 负责“写出答案”。
所以它特别适合翻译、摘要、问答、改写、结构化信息转自然语言等任务。

6.1 BART：把文本弄乱，再让模型还原

BART 的训练思路类似 去噪自编码器 Denoising Autoencoder 。

它会先把原文破坏掉，比如：

破坏方式	含义
Token Masking	遮住部分词
Token Deletion	删除部分词
Sentence Permutation	打乱句子顺序
Document Rotation	旋转文档
Text Infilling	挖掉一段连续文本

然后让模型恢复原文。

通俗解释：

BART 像是在训练一个“文本修复师”。
给它一段被破坏、被打乱、缺字少句的文本，让它恢复成原来的样子。

这让 BART 同时具备理解和生成能力，所以它既能做分类，也能做生成任务。

6.2 T5：把所有任务都变成 Text-to-Text

T5 的核心思想非常重要：

所有 NLP 任务都可以统一成：文本输入 → 文本输出

比如：

任务	输入	输出
翻译	translate English to German: That is good.	Das ist gut.
情感分类	cola sentence: The course is jumping well.	not acceptable

任务	输入	输出
摘要	summarize 🔗: 一大段新闻	一句话摘要
问答	question + context	answer

通俗解释：

T5 把所有任务都变成“给我一句话，我再回你一句话”。
不管是分类、翻译、摘要还是问答，本质都变成文本生成。

这和 BERT 的思路不同：

模型	分类任务怎么做
BERT	用 [CLS] 的 embedding 接一个分类头
T5 / GPT 类生成模型	直接生成自然语言标签或答案

例如情感分类不再是输出类别编号，而是直接生成：

positive

或者：

negative

7. 优化算法：怎么把模型真的训练出来？

讲义第二大部分讲优化算法。

定义目标函数只是第一步，真正训练大模型时，还要解决三个问题：

问题	含义
收敛速度	怎么让模型更快学到好结果
数值稳定	怎么避免训练发散、梯度爆炸/消失
内存效率	怎么在有限 GPU 显存里训练大模型

通俗解释：

小模型训练像普通健身，控制好动作就行。
大模型训练像造火箭，任何一个参数、精度、学习率、初始化不对，都可能直接炸。

8. SGD：最基础的训练方式

最基础的方法是 随机梯度下降 SGD：

- 1. 初始化参数
- 2. 随机抽一批训练样本 mini-batch
- 3. 计算 loss
- 4. 计算梯度
- 5. 沿着让 loss 下降的方向更新参数
- 6. 不断重复

通俗解释：

模型每次看一小批题，算自己错了多少，然后根据错误修正参数。
一轮一轮下来，模型逐渐变好。

但是大语言模型通常不会只用最朴素的 SGD，因为它收敛慢，也不够稳定。

9. Adam：大模型训练中的常用优化器

讲义重点介绍了 Adam。

Adam 的两个核心思想：

机制	通俗解释
Momentum 动量	如果过去一直往某个方向改，现在也继续保持这个趋势
Adaptive step size 自适应步长	不同参数用不同学习率，有的参数多改一点，有的少改一点

通俗解释：

SGD 像是每一步都只看当前坡度。
Adam 像是既看当前坡度，也记得之前往哪里走比较有效，还会给不同方向分配不同步子大小。

这让 Adam 在深度学习里非常常用。

但 Adam 的问题是： **更耗显存**。

因为它除了存参数和梯度，还要存一阶动量和二阶动量。

10. AdaFactor：为了省显存

Adam 会为每个参数都保存额外状态，模型越大，优化器状态越占内存。

AdaFactor 的思路是：

不要完整保存矩阵的动量信息，而是只保存行统计和列统计，再近似还原。

通俗解释：

Adam 像是给每个参数都单独记账。

AdaFactor 像是只记“每一行”和“每一列”的汇总账，节省大量空间。

讲义提到，AdaFactor 被用于训练 T5。

不过它也不是银弹，实际训练起来可能比较难调。

11. 混合精度训练：用 FP16 省显存

默认训练使用 FP32，也就是 32 位浮点数。

混合精度训练会大量使用 FP16，也就是 16 位浮点数，以减少显存占用。

好处是：

显存约减半，速度也可能更快。

但问题是：

FP16 表示范围更小，太小的数可能直接变成 0。

所以常见做法是：

做法	含义
Master weights 用 FP32 保存	保留一份高精度主参数

做法	含义
前向/反向大量用 FP16	节省显存和计算
Loss scaling	把 loss 放大，避免梯度太小变成 0

通俗解释：

混合精度就像日常计算用草稿纸，关键账本用正式账本。
大部分计算用低精度省资源，但核心参数仍然用高精度保存，避免算坏。

12. 学习率：Transformer 需要 Warmup

通常我们以为学习率应该一开始大，然后慢慢变小。

但 Transformer 训练中，常常需要 **learning rate warmup**：

一开始学习率从小逐渐增大，然后再慢慢下降。

为什么？

讲义提到，一个可能原因是和 LayerNorm、Adam 稳定性有关。如果一开始学习率太大，模型还没稳定，就可能训练崩掉。

通俗解释：

训练 Transformer 就像开跑车。
不能一脚油门踩到底，要先慢慢加速，等车稳定后再进入正常速度。

13. 初始化：大模型参数不能随便设

讲义提到标准初始化方法，比如 Xavier initialization。

但 GPT-2、GPT-3、T5 还会做额外缩放。

例如 GPT-2 / GPT-3 会根据残差层数量额外缩放权重。

通俗解释：

大模型层数很多，每一层都会对信号产生影响。
如果初始化太大，信号可能越传越爆；如果太小，信号可能越传越弱。
所以初始化必须非常小心。

14. GPT-3 的训练细节

讲义最后列出 GPT-3 的一些训练配置：

配置	含义
Adam 参数 $\beta_1=0.9$, $\beta_2=0.95$, $\epsilon=1e-8$	Adam 优化器设置
Batch size 约 320 万 token	每次训练看大量 token
Gradient clipping	防止梯度爆炸
前 3.75 亿 token 做 learning rate warmup	先慢慢提高学习率
Cosine learning rate decay	后期平滑降低学习率
逐渐增大 batch size	训练过程中扩大批量
Weight decay 0.1	正则化，防止过拟合
Mixed precision training	省显存、提速

通俗解释：

GPT-3 不是简单地“数据多 + 模型大”就能训练出来。
它背后有一整套工程技巧：学习率怎么变、batch 怎么增、梯度怎么裁剪、精度怎么混合、参数怎么初始化。
这些细节决定了训练能不能稳定跑完。

15. 这份讲义最重要的思想

我觉得这份 Training 讲义真正想传达的是三层意思。

第一，不同模型架构对应不同训练目标。

GPT 学的是：

根据前文预测下一个 token。

BERT 学的是：

根据完整上下文补全被遮住的 token，并理解句子关系。

T5 / BART 学的是：

把一种文本转换成另一种文本。

第二， 预训练目标决定模型能力倾向 。

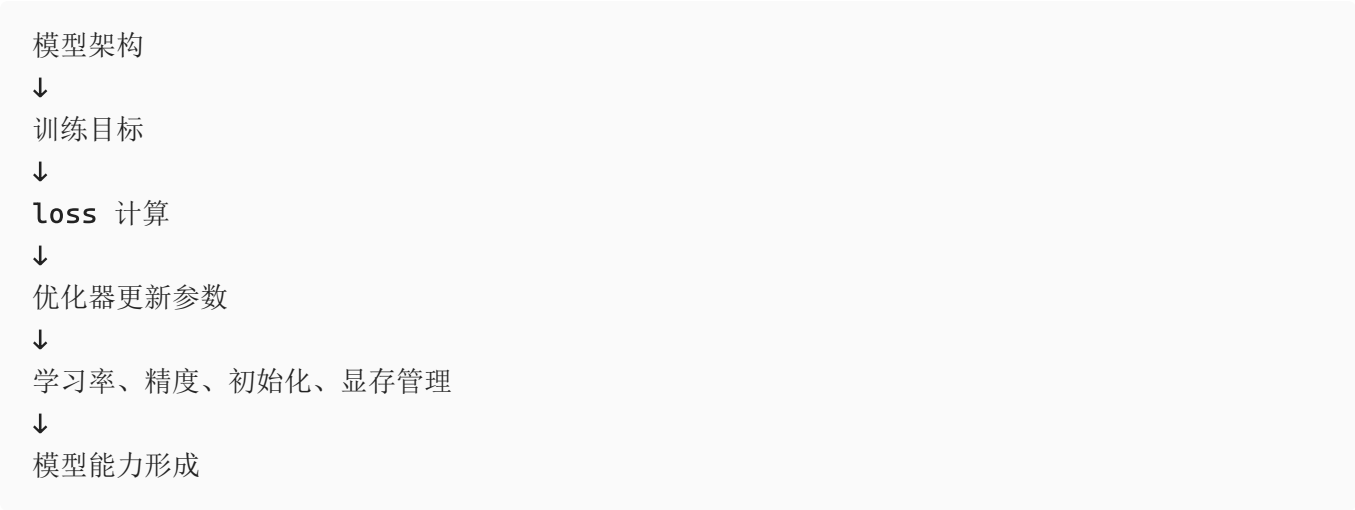
训练目标	更容易形成的能力
下一个 token 预测	生成、续写、对话
Masked LM	理解、分类、匹配
Denoising / Text-to-text	生成、摘要、翻译、改写、问答

第三， 大模型训练是算法 + 工程的结合 。

训练目标只是数学上定义“要学什么”；优化算法和工程技巧决定“能不能真的学出来”。

16. 和你现在学习大模型的关系

如果你想理解大模型训练，这份讲义可以帮你建立一个非常重要的框架：



更通俗地说：

- 架构决定模型长什么样；
- 训练目标决定它练什么题；
- 数据决定它见过什么世界；
- 优化算法决定它怎么改正错误；
- 工程技巧决定它能不能稳定训练到最后。

这也是理解 GPT、BERT、T5、BART 以及后续大模型训练范式的基础。